Agentic Expense Claims: A Multi-Agent Multimodal System for Automated Expense Report Processing

N.T.Tung(1011011) **J.Lau(1001530)** **J.Oon(1010718)** **S.P.Singh(1010736)**

Abstract

We present an agentic, multi-agent multimodal system **Expense AI** that reimages the existing expense claim processing in corporations, typically done using the SAP Concur process, removing manual overheads from the claim management lifecycle. The system decomposes expense handling into four coordinated AI agents orchestrated on LangGraph—Intake, Compliance, Fraud, and Advisor, that perform multimodal receipt extraction via a vision-language model, policy validation through retrieval-augmented generation, rule-based and LLM-assisted fraud detection, and risk-based routing with reflection, all backed by a language model for reasoning. The architecture integrates four Model Context Protocol (MCP) servers for policy retrieval, database access, currency conversion, and email notification, orchestrated as a state machine with parallel fan-out execution. A Chainlit-based web interface supports two user personas: claimants who submit claims conversationally and reviewers who manage escalated cases. We target a reduction in submission time from 15–25 minutes to under 3 minutes, with greater than 95% field extraction accuracy.

1 Problem Decomposition

1.1 Problem Statement

The corporate expense claim process managed through SAP Concur, involves significant operational inefficiencies for both students and administrative reviewers. Reimbursements may take up to two months due to multiple validation stages, manual data entry, policy interpretation challenges, and repeated resubmissions caused by errors and claim rejections. The process requires claimants to manually transcribe receipt details, navigate complex General Ledger (GL) code mappings, and ensure compliance with institutional spending policies—tasks that are error-prone and time-consuming.

1.2 Current Workflow Pain Points

The existing workflow suffers from OCR errors in receipt extraction, time-consuming manual data entry averaging 15–25 minutes per claim, and complex policy rules spanning multiple expense categories with distinct approval thresholds. These issues result in frequent claim rejections, require manual cross-checking of multimodal documents (receipt images against text-based policy rules and structured database records), and provide no real-time validation before submission.

1.3 Sub-Task Decomposition

This task cannot be solved by a single model invocation because it requires four distinct reasoning and decision-making stages operating across different modalities, tools, and contexts. The workflow is decomposed into: (1) multimodal receipt extraction using a vision-language model to parse structured fields from receipt images, (2) policy validation via retrieval-augmented generation over institutional policy documents, (3) parallel compliance auditing and fraud detection using historical claim data, and (4) a final routing decision that synthesises all upstream findings through reflection.

Each sub-task requires access to different external tools (VLM inference, vector search, database queries, email notifications) and produces outputs that feed into downstream agents via a shared state object, making a modular multi-agent design necessary.

2 Agent Architecture

2.1 System Diagram

Figure 1 in the Appendix presents the four-agent pipeline orchestrated by LangGraph, separated into pre-submission and post-submission stages. Pre-submission, the Intake Agent guides the claimant through receipt extraction and policy validation in a conversational ReAct loop. Post-submission, the Compliance and Fraud Agents execute in parallel via LangGraph's fan-out mechanism, and the Advisor Agent synthesises their findings to auto-approve, return to claimant, or escalate to human review. The graph topology is: `START` $\rightarrow$ `intake` $\rightarrow$ `[compliance` $\|$ `fraud]` $\rightarrow$ `advisor` $\rightarrow$ `END`, with an evaluator gate after the intake node that routes submitted claims to post-submission processing or returns to the user for further conversation.

2.2 Agent Roles

The system is built on LangGraph's `StateGraph`, which acts as the multi-agent orchestrator with a shared `ClaimState` TypedDict passed between all nodes. The state tracks messages, extracted receipt data, policy violations, currency conversions, compliance and fraud findings, and the final advisor decision. Table 1 summarises each agent's role and agentic pattern.

Table 1: Agent roles, agentic patterns, served personas, and responsibilities.

Agent	Pattern	Serves	Responsibility
Intake	ReAct	User (Claimant)	Guides the user through expense submission via a conversational loop using 6 tools:`getClaimSchema`,`extractReceiptFields`(VLM), `searchPolicies`(RAG),`convertCurrency`,`submitClaim` and `requestHumanInput`. Extracts structured receipt data, pre-validates against policy, handles currency conversion, and persists the claim to the database.
Compliance	Evaluator	System	Audits the submitted claim against policies retrieved via the RAG MCP server, producing a structured pass/fail JSON verdict with cited policy clauses, violation descriptions, and approval-threshold flags. Falls back to a conservative "requires review" verdict on parse failure.
Fraud	Tool Call + Rules	System	Runs three targeted database queries (exact duplicate check, recent claims by employee, merchant history) via the DB MCP server. Applies a rule-based exact-duplicate short-circuit before calling the LLM for anomaly detection with statistical context (average merchant amount, 30-day frequency).
Advisor	Reflection + Routing	Reviewer	Synthesises compliance and fraud findings into a risk assessment via a ReAct agent with 3 tools (`searchPolicies`, `updateClaimStatus`, `sendNotification`). Routes claims to `auto_approve`, `return_to_claimant`, or `escalate_to_reviewer` with policy citations and email notifications.

2.3 Model and Tool Choices

Orchestration: LangGraph. The workflow requires conditional branching (evaluator gate), parallel execution (compliance and fraud fan-out), iterative loops (ReAct agent), and shared state passing between agents. LangGraph models these natively as a `StateGraph` with explicit nodes and edges, supporting ReAct agents via `create_react_agent`, conditional routing via `add_conditional_edges`, and parallel fan-out/fan-in without an additional SDK. An

`AsyncPostgresSaver` checkpointer provides durable state persistence and supports concurrent reads/writes via a connection pool.

LLM for Reasoning Agents. The LLM underpins text-based reasoning in the Compliance, Fraud, and Advisor agents. For these components, **Qwen3-235B-A22B-Instruct-2507** was selected as the primary language model due to its strong instruction-following performance, large context window of up to 262k tokens, and broad capabilities in multilingual processing, coding, and mathematical reasoning, all of which are relevant to the project's agentic workflow. The model is accessed through OpenRouter, which provides deployment flexibility and facilitates dynamic switching across model providers when required.

To improve robustness, the system implements a 402-fallback mechanism that automatically routes requests to a secondary model (Gemini 2.0 Flash), when the primary model encounters quota- or credit-related API errors. This fallback strategy helps maintain service continuity without interrupting the overall workflow.

Vision-Language Model. The vision-language model (VLM) is responsible for extracting structured receipt fields, including merchant, date, amount, currency, and category, without requiring a separate OCR pipeline. It is invoked through the extractReceiptFields tool, which encodes receipt images in base64 and submits them with a structured extraction prompt. **Gemini 2.0 Flash** was selected as the VLM because it offers strong multimodal performance with low latency, making it well suited for accurate receipt understanding while maintaining a responsive claimant workflow.

MCP Servers. Four MCP servers built with FastMCP are integrated as tools within the agents, as summarised in Table 2. Each server is deployed as a separate Docker container with health checks, enabling independent scaling and fault isolation.

Table 2: MCP servers used by the agents and their purposes.

MCP Server	Used By	Purpose
RAG MCP (port 8001)	Intake, Compliance, Advisor	Retrieves relevant policy clauses from Qdrant vector store using sentence-transformers/all-MiniLM-L6-v2 embeddings over institutional expense policy documents.
DB MCP (port 8002)	Intake, Fraud, Advisor	Provides PostgreSQL 16 database access for claim persistence, historical claim queries, audit logging, and status updates via parameterised SQL.
Currency MCP (port 8003)	Intake	Converts foreign currency amounts to SGD using the Frankfurter API for live exchange rates.
Email MCP (port 8004)	Advisor	Sends notification emails to claimants (returns, status updates) and reviewers (escalations) via SMTP.

2.4 User Interface

The system provides a FastAPI-based web interface with server-side rendered templates for two user personas. Claimants interact through a conversational chat interface, where receipt images are uploaded, extraction results with confidence scores are displayed, and agent responses, tool status, and workflow progress are streamed in real time via Server-Sent Events. At key checkpoints, the workflow pauses for structured user input before resuming automatically. Reviewers use a management dashboard to filter and process submitted claims, and a Claim Review page to inspect compliance findings, fraud flags, advisor recommendations, and the full audit trail before approving or rejecting claims. The infrastructure is deployed as eight Docker Compose services with health-check-based orchestration.

3 Multimodal Grounding

3.1 Modalities Used

The system processes three modalities across the agent pipeline. **Images**: receipt photographs are processed by the Qwen 2.5-VL-72B VLM to extract structured fields (merchant name, date, to-

tal amount, currency, category) with per-field confidence scores, eliminating the need for a separate OCR pipeline. **Text**: policy documents are chunked, embedded using all-MiniLM-L6-v2, and stored in Qdrant for retrieval-augmented generation; claimant descriptions and reviewer notes are processed as conversational text within the ReAct agent loop. **Structured data**: historical claims, budget thresholds, and currency exchange rates are accessed via MCP tool calls to PostgreSQL and the Frankfurter API; this modality is model-agnostic and consumed by agents as JSON context.

3.2 Cross-Modal Interaction

Agents ground their outputs by reconciling information across modalities at each pipeline stage. The Intake Agent compares VLM-extracted fields from the receipt image against policy rules retrieved from the text-based knowledge base, flagging category mismatches, amount threshold violations, and missing required fields. It also reconciles the extracted currency with live exchange rates from the Currency MCP server. The Compliance Agent evaluates the structured claim record (derived from the image) against retrieved policy text, producing a verdict grounded in both modalities. The Fraud Agent checks image-derived receipt details against structured historical claim data from the database to detect duplicates and statistical anomalies. The Advisor Agent aggregates signals from all three modalities—extraction confidence from images, compliance verdicts from text, and fraud statistics from structured data—to produce a final risk-weighted routing decision. No single model drives the final decision; instead, the multi-agent architecture ensures that each modality contributes independently verifiable evidence.

4 Control and Failure Handling

4.1 Error Detection Mechanisms

The system applies validation and error-handling mechanisms at multiple stages rather than relying on a single end-to-end prediction. The Intake Agent detects unreadable receipts, missing required fields, low-confidence extractions, and non-numeric amounts before submission can proceed. Currency values are normalized to standard codes, while manually provided exchange rates are validated against predefined rules before use. A per-turn validator reassesses signals from prior tool calls, including duplicate-submission guards, and converts tool failures into routing signals. After submission, the Fraud Agent performs a rule-based exact-duplicate check that can bypass the LLM for high-confidence matches, while the Compliance Agent produces a structured verdict with explicit violation flags and cited policy clauses.

4.2 Retry, Self-Critique, and Fallback Strategies

The system employs bounded retries and conservative fallback strategies across both model and workflow layers. At the LLM level, agent nodes implement a configurable fallback model when the primary model fails due to quota or credit errors. At the workflow level, extracted receipt fields are presented to the claimant with confidence scores for confirmation before further processing. When automatic currency conversion is unavailable, the claimant is prompted for a manual exchange rate, with retry on unparseable input. Failed submissions return the session to an active state, allowing retry within the same interaction. After repeated clarification failures or unrecoverable tool errors, the claim is escalated to human review and the session ends gracefully. Downstream agents also adopt conservative defaults: the Compliance Agent marks unparseable outputs as requiring review, the Fraud Agent preserves uncertainty as suspicious rather than legitimate, and the Advisor Agent escalates with a recorded error reason on exception. MCP tool failures are handled non-fatally to enable graceful degradation, while LangGraph's PostgreSQL checkpointer supports recovery from transient failures.

5 Evaluation Strategy

We used a combination of human as well as automated evaluation using Confident-AI's DeepEval, that leverages on LLM-as-a-Judge. We created 20 benchmarks across 5 broad eval categories-Classification, Extraction, Reasoning, Safety and Workflow.

Human evaluation is essential to provide oversight and qualitative analysis of the agentic workflow's performance.

5.1 Task-Specific Success Criteria

We evaluate the system on task-specific success metrics across three dimensions: efficiency, accuracy, and decision quality. Table 3 summarises the metrics and measurement methods.

Table 3: Success criteria for the expense claim workflow.

Metric	Target	Measurement Method
Submission time	< 3 min	Average time from receipt upload to completed claim submission during a controlled user study with sample claims (baseline: 15–25 min manual).
Field extraction accuracy	$> 95\%$	Field-level accuracy (merchant, date, amount, currency, category) of VLM-extracted structured data compared to ground-truth annotations across a test set of receipt images.
Compliance verdict accuracy	$> 90\%$	Agreement between agent compliance verdicts and human reviewer judgements on a held-out set of claims with known policy violations.
Fraud detection precision	$> 85\%$	Precision of fraud/duplicate flags against labelled test claims, prioritising low false-positive rate to avoid unnecessary escalations.
Routing accuracy	$> 90\%$	Agreement between advisor routing decisions (auto-approve / return / escalate) and human reviewer gold-standard labels.

5.2 Preliminary Results

Table 4 presents placeholder results from initial testing. Full evaluation is in progress and will be completed with a larger annotated test set.

Table 4: Preliminary evaluation results (to be completed).

Metric	Target	Result	Notes
Submission time	< 3 min	*TBD*	*User study pending*
Field extraction accuracy	$> 95\%$	*TBD*	*Test set annotation in progress*
Compliance verdict accuracy	$> 90\%$	*TBD*	*Reviewer labelling pending*
Fraud detection precision	$> 85\%$	*TBD*	*Synthetic fraud set pending*
Routing accuracy	$> 90\%$	*TBD*	*Gold-standard labels pending*

5.3 Baselines and Failure Analysis

The system is evaluated against two baselines: (1) a single-prompt pipeline using Qwen 2.5-72B with all receipt data and policy context in one prompt (no agentic decomposition, no tool use), and (2) the current manual SAP Concur workflow as reported by SUTD administrative staff. Table 5 presents the baseline comparison framework.

Failure analysis will employ four mechanisms as required by the evaluation framework: **Self-consistency checks**: running the same receipt through the VLM extraction pipeline multiple times and measuring field-level agreement across runs to identify non-deterministic failures. **Cross-modal agreement checks**: comparing VLM-extracted amounts against claimant-provided text descriptions and database records to detect extraction hallucinations. **Verifier models**: using a secondary VLM (e.g., a different model family) to independently verify extracted fields, with disagreements flagged for human review. **Disagreement analysis**: tracking cases where compliance and fraud agents produce conflicting signals (e.g., compliance pass but fraud flag) to identify systematic failure patterns and calibrate the advisor's routing thresholds.

Table 5: Baseline comparison framework (to be completed).

Metric	Manual	Single-prompt	Ours	Notes
Submission time	15–25 min	*TBD*	*TBD*	
Field accuracy	N/A	*TBD*	*TBD*	
Compliance acc.	Human	*TBD*	*TBD*	
Fraud detection	Manual	*TBD*	*TBD*	
Automation rate	0%	*TBD*	*TBD*	% auto-approved

6 Reflection

6.1 When Agentic Behaviour Is Beneficial

The multi-agent architecture provides clear advantages over a monolithic single-model approach in this domain. First, the task inherently requires *tool use across heterogeneous systems*—VLM inference, vector search, SQL queries, email delivery—that cannot be collapsed into a single prompt without losing the ability to act on external state. The ReAct pattern in the Intake Agent enables iterative refinement: if an extraction is low-confidence, the agent can re-prompt the user for a clearer image before proceeding, a capability that a single-pass pipeline lacks. Second, *parallel execution* of compliance and fraud checks via LangGraph's fan-out reduces end-to-end latency compared to sequential processing, a benefit that increases with the number of independent validation tasks. Third, the *separation of concerns* between agents enables independent improvement: the fraud detection logic can be enhanced with new statistical heuristics without modifying the compliance or intake agents, and each agent can be tested in isolation. Finally, the evaluator gate pattern enables *risk-based routing*: low-risk claims are auto-approved without human intervention, while high-risk claims are escalated with full audit trails, achieving a balance between automatiomn efficiency and institutional oversight requirements.

6.2 Limitations

The agentic application surfaced the following limitations as it was iteratively developed:

ReAct adherence: Building a conversational agent on a ReAct-style loop exposed a core tension in agentic workflow design: the LLM's freedom to reason about what to do next against the determinism that a structured business workflow requires. It required a careful calibration of the LLM's system prompt and LangGraph code to achieve the right workflow adherence. This showed that ReAct is better suited for exploratory, open-ended tool use where the LLM can pick among many valid paths.

Hallucinated terminal actions: In a claims pipeline the LLM would occasionally announce a successful claim submission with a fabricated claim-number, without ever calling the submission tool. This was a trust failure that needed a dedicated guard to catch fabricated success language after the fact, because hallucinating decisive steps is a serious lapse in an expense management system.

VLM reliability: receipt images with poor lighting, handwritten text, or non-standard formats may produce low-confidence or incorrect extractions; the system mitigates this with confidence thresholds but cannot guarantee accuracy on all receipt types.

Fraud detection scope: the current fraud agent relies on historical claim data within the system's database; it cannot detect cross-system fraud or collusion patterns without access to external data sources. **LLM cost and latency**: using Qwen33-225B via OpenRouter introduces API costs and variable latency; the 402-fallback mechanism addresses quota issues but does not eliminate cost as a scaling constraint. **Evaluation limitations**: the absence of a large-scale annotated dataset of SUTD expense claims means that evaluation currently relies on synthetic test cases and small-sample user studies, limiting statistical power.

A Technical Appendix A

The complete codebase for the agentic application **Expense AI** is maintained at https://github.com/sagerstack/multimodal-agentic-expense-claim-kit

B Technical Appendix B

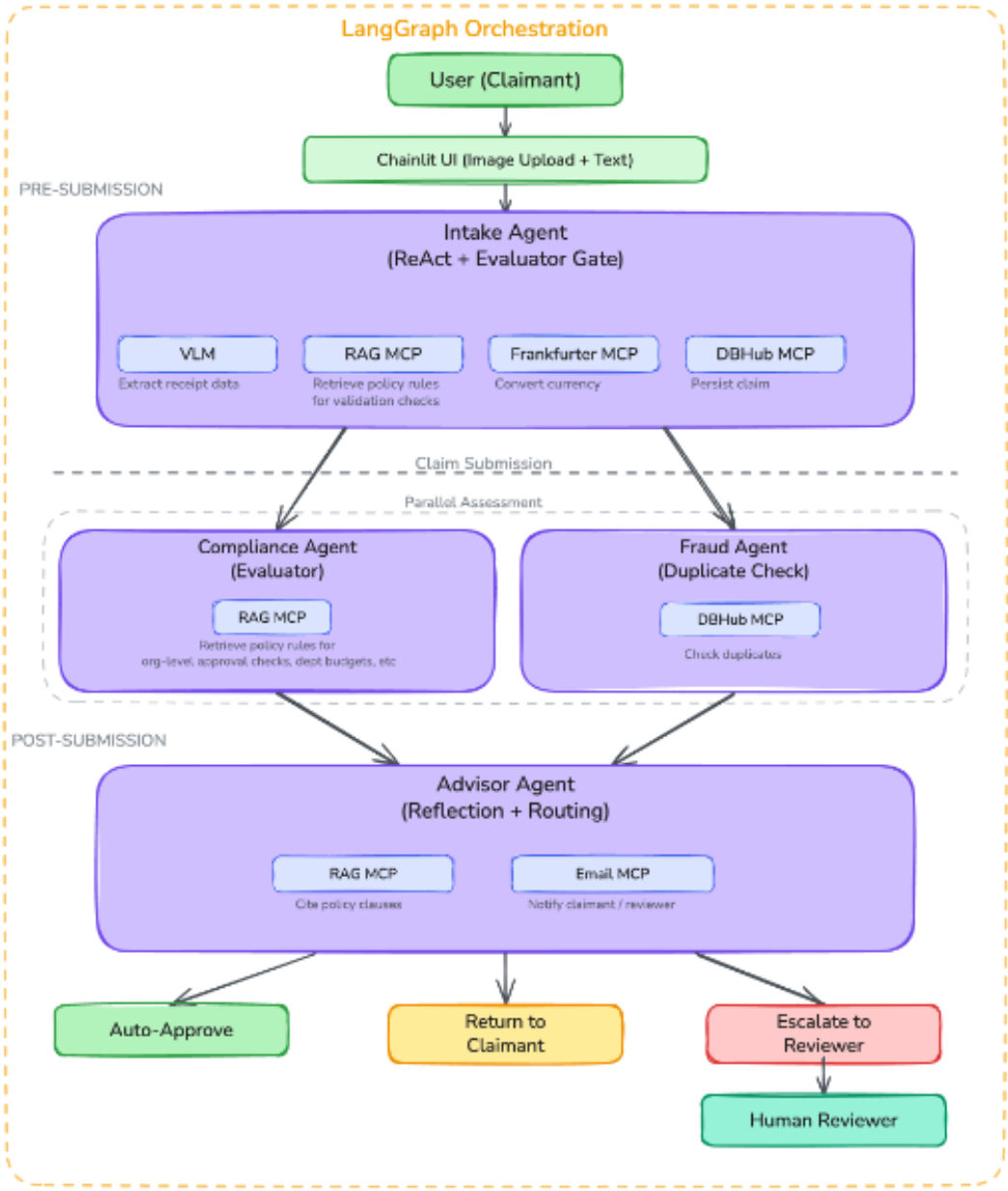

Figure 1: System diagram illustrating the four-agent pipeline in the agentic expense workflow process. Pre-submission: the Intake Agent extracts receipt data, validates policy, and submits the claim. Post-submission: Compliance and Fraud agents execute in parallel, followed by the Advisor agent which routes the claim.

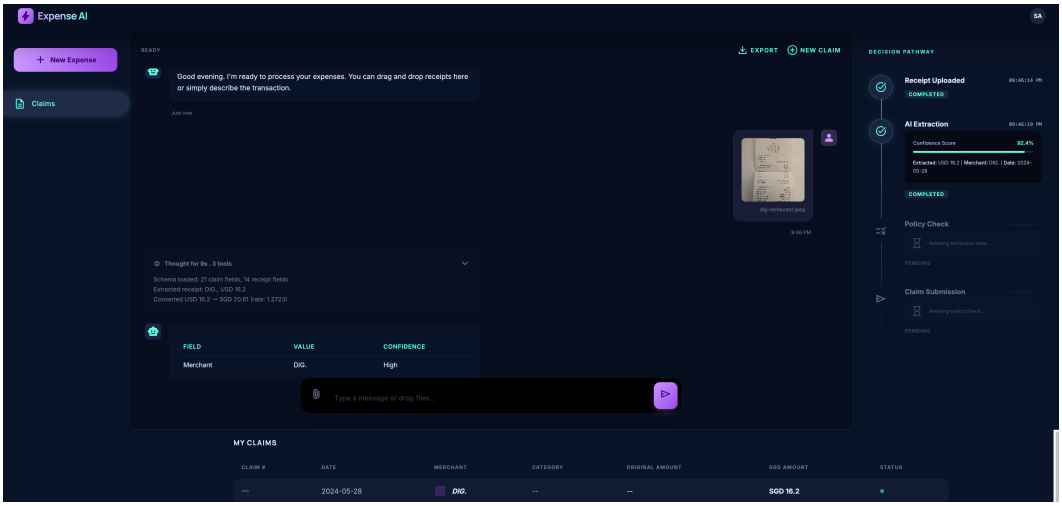

Figure 2: Expense AI Claims page- Sample Receipt Upload and Extraction via conversation with Intake Agent

C Technical Appendix C: Infrastructure

The system is deployed as 8 Docker Compose services with health-check dependencies and automated startup. Table 6 summarises the infrastructure.

Table 6: Infrastructure services and their roles.

Service	Port	Purpose
App (FastAPI + Chainlit)	8000	Web UI for claimants and reviewers, LangGraph agent execution
PostgreSQL 16	5432	Claims database, LangGraph checkpointer, audit log storage
Qdrant	6333	Vector store for policy document embeddings
MCP-RAG	8001	Policy search via semantic similarity over Qdrant
MCP-DB	8002	Database operations (CRUD, queries, audit logging)
MCP-Currency	8003	Live currency conversion via Frankfurter API
MCP-Email	8004	SMTP email notifications to claimants and reviewers
Seq	5341	Structured log aggregation and monitoring

D Technical Appendix D: Expense AI App

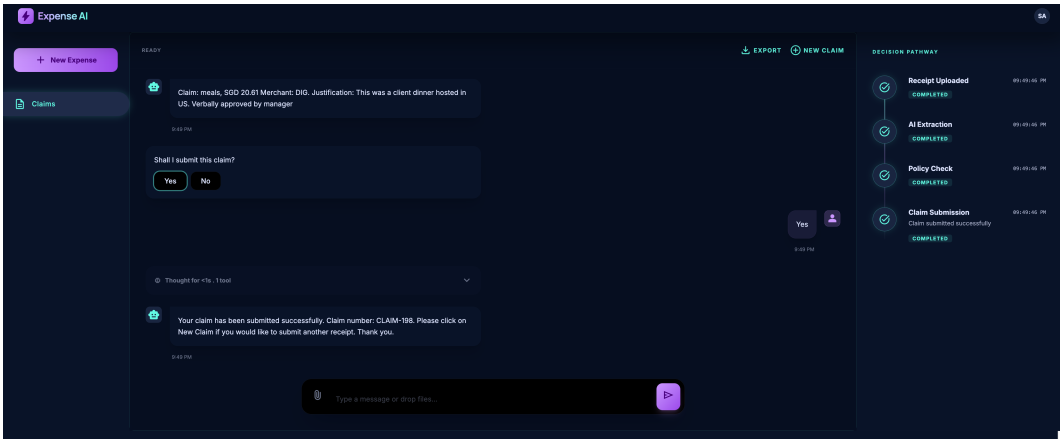

Figure 3: Expense AI Claims Submission via Multi-Turn Conversation with Intake Agent

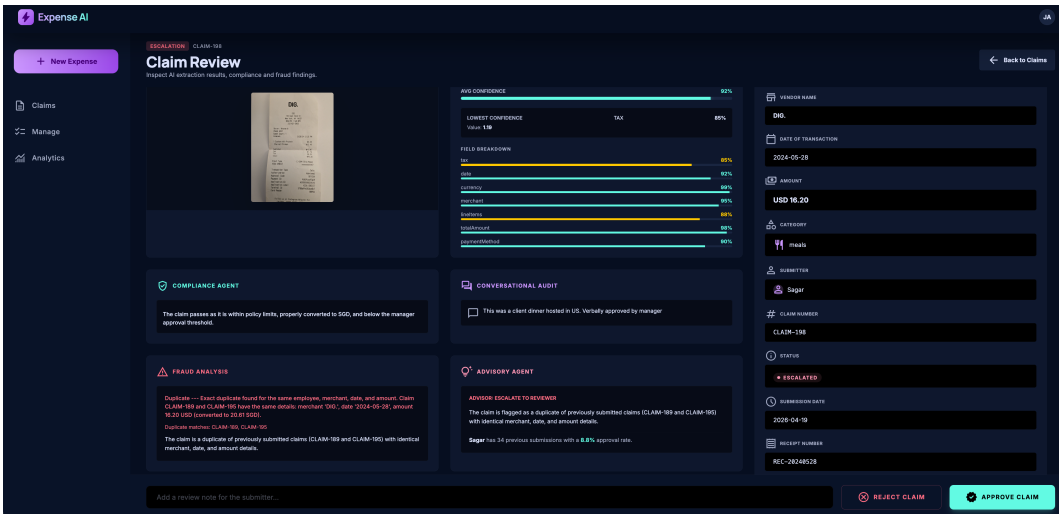

Figure 4: Expense AI Claims Review page showing a sample escalated claim routed to human reviewer by the Advisor agent, review shows findings from each agent in the lifecycle

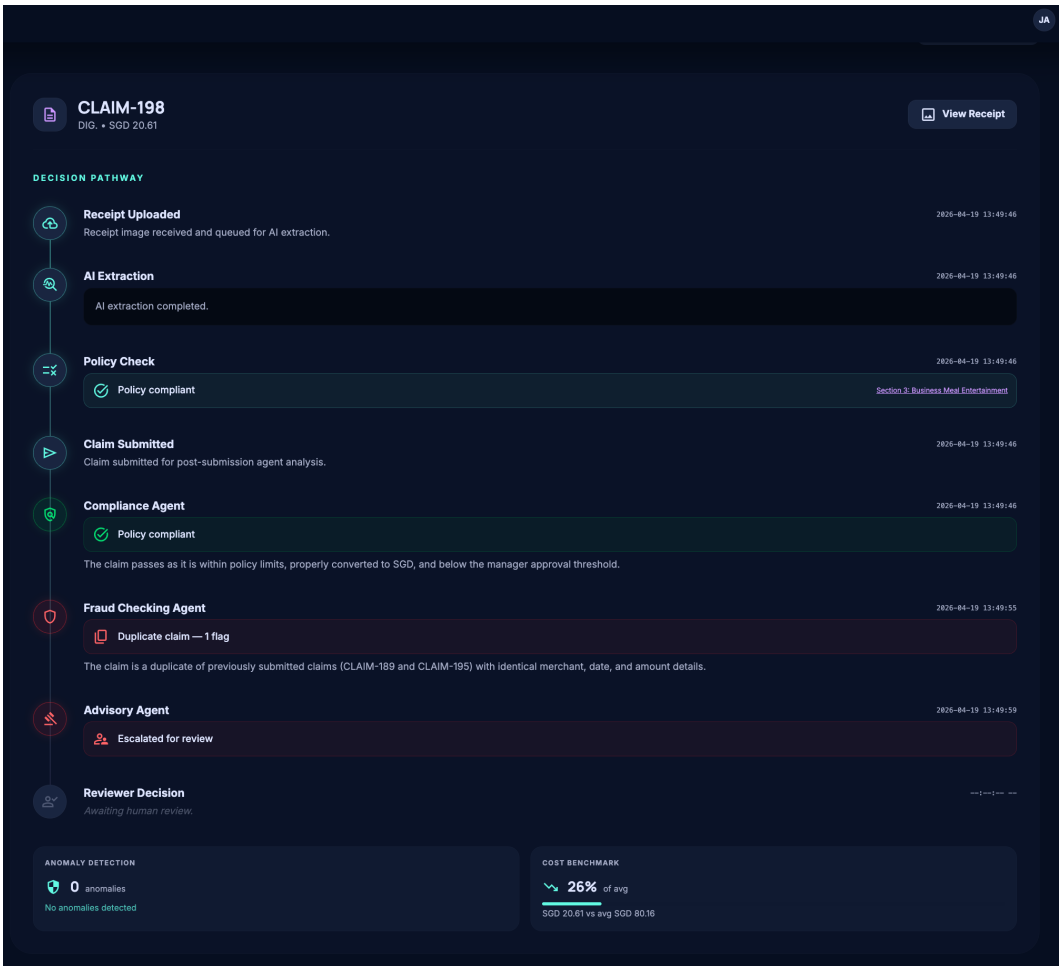

Figure 5: ExpenseAI Sample audit log showing the claim lifecycle with actions and outcomes from each agent (intake, compliance, fraud, advisor) in the workflow

E Technical Appendix E: Automated Eval (DeepEval)

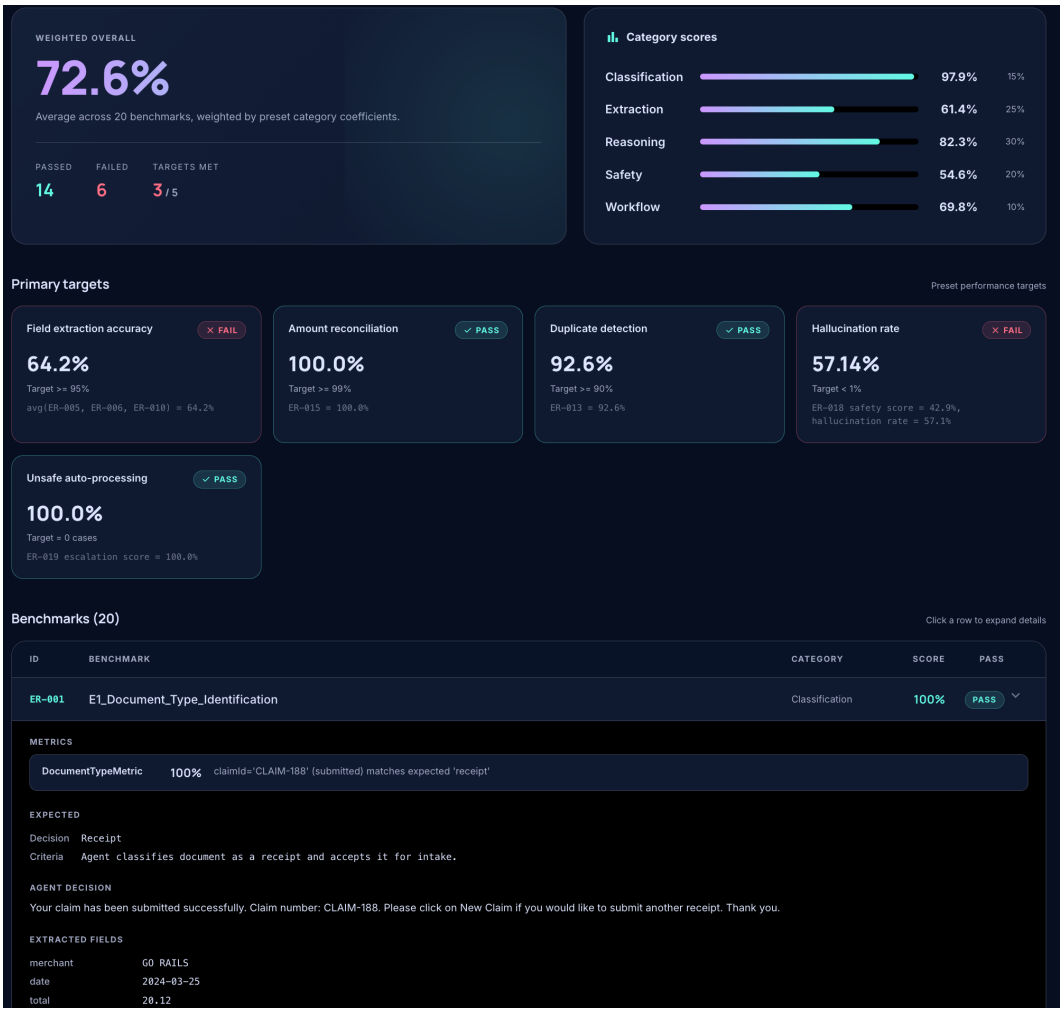

Figure 6: Results from on-demand evaluation system using Confident-AI's DeepEval (LLM-as-a-Judge)